

GestorDeContratos

Flujo completo: método de pago integrado

Este documento resume el diseño y flujo del sistema de pagos integrado para la plataforma GestorDeContratos (React + Django). Cubre desde la generacion automatica de cuotas hasta la actualizacion de estado tras el pago del inquilino, sin necesidad de login.

1. Arquitectura general

El sistema esta compuesto por cuatro capas bien separadas. Cada una tiene una responsabilidad clara y se comunica con la siguiente de forma asincrona o via webhook.

Capa	Responsabilidad
Frontend React	Vista del inquilino: panel de cuotas, boton de pago, tokenizacion de tarjeta via SDK del proveedor.
Backend Django	Validacion del contrato, generacion de tokens, llamada al proveedor de pagos, registro en base de datos.
Proveedor externo	Mercado Pago / Stripe: procesa el cobro, devuelve resultado y confirma via webhook.
Persistencia	PostgreSQL via Django ORM: modelos Contrato, Cuota y PagoRegistro. Celery para tareas periodicas.

2. Flujo de pago (vista general)

Cuando un inquilino paga desde la aplicacion, el flujo atraviesa las cuatro capas en este orden:

Paso	Detalle
1. Selecciona contrato	El inquilino ve su panel con monto, vencimiento e historial de cuotas.
2. Tokenizacion	El SDK del proveedor (browser) tokeniza los datos de tarjeta. El numero nunca llega al servidor.
3. POST /api/pagos/iniciar/	Django recibe el token y valida que el contrato este activo y vigente.
4. Llamada al proveedor	Django llama server-to-server a la API del proveedor con el token y el monto.
5. Respuesta	Si el pago es aprobado, se registra en DB. Si es rechazado, se notifica al frontend.
6. Webhook	El proveedor confirma el estado final via webhook. Django actualiza la cuota a 'pagado'.
7. Respuesta al frontend	Se devuelve recibo y nuevo estado del contrato al inquilino.

3. Validaciones antes de procesar el pago

Django realiza estas verificaciones antes de llamar al proveedor:

Contrato activo	El contrato debe estar en estado 'activo'. No se procesan contratos vencidos o rescindidos.
Cuota pendiente	Solo se puede pagar una cuota en estado 'pendiente'. Las ya pagadas se ignoran.
Token valido	En el flujo sin login, el token debe existir y no haber expirado (15 dias por defecto).
Monto correcto	El monto enviado desde el frontend debe coincidir con el registrado en la cuota.

4. Flujo sin login: link por WhatsApp o email

Para 50-200 inquilinos, la mejor experiencia es recibir un link directo sin necesidad de registrarse. El sistema genera un token unico por cuota y lo envia automaticamente cada mes.

Paso	Detalle
1. Generacion automatica	Celery beat corre el 1 de cada mes y genera las cuotas del periodo.
2. Token unico por cuota	Django genera un UUID + fecha de expiracion (15 dias). Se guarda en el modelo Cuota.
3. Envio del mensaje	Twilio (WhatsApp) o SendGrid (email) envia el link al inquilino.
4. Formato del mensaje	'Hola Juan, tu alquiler de mayo vence el 10/05. Pagar: tuapp.com/pagar/abc123'
5. Pagina publica	La vista /pagar// no requiere login. Muestra monto, contrato y boton de pago.
6. Pago y actualizacion	Tras el pago, el webhook marca la cuota como pagada e invalida el token.

5. Que ve el inquilino

La pantalla del inquilino tiene tres secciones claras:

Resumen del contrato	Direccion, fecha de vencimiento del contrato, inquilino.
Metricas clave	Monto actual del alquiler / Proximo vencimiento / Cuotas pagas sobre el total.
Alerta de pago	Banner visible cuando hay una cuota pendiente proxima a vencer.
Historial de cuotas	Lista con estado (Pendiente / Pagado / Vencido), monto, fecha y boton de pago.
Estados posibles	Pendiente (amarillo), Pagado (verde), Vencido (rojo). El estado cambia automaticamente.

6. Actualizacion automatica de estado

El sistema no requiere intervencion manual para actualizar estados. Hay dos mecanismos:

Webhook del proveedor	Cuando el pago es aprobado, el proveedor llama a /api/pagos/webhook/. Django actualiza la cuota a 'pagado', registra fecha y monto, e invalida el token.
-----------------------	--

Celery beat diario	Todos los días a las 00:00, una tarea revisa cuotas con fecha_vencimiento < hoy y estado='pendiente', y las marca como 'vencido'.
Proximo vencimiento	Al marcar una cuota como pagada, Django calcula y actualiza el campo proximo_vencimiento del contrato segun la frecuencia pactada.

7. Modelos de datos sugeridos (Django)

Estructura basica recomendada para los modelos en Django:

Contrato: inquilino (FK User), direccion, monto_actual, fecha_inicio, fecha_fin, frecuencia_pago, proximo_vencimiento, estado (activo/rescindido/vencido).
Cuota: contrato (FK), periodo (ej: '2025-05'), monto, fecha_vencimiento, fecha_pago, estado (pendiente/pagado/vencido), token, token_expira.
PagoRegistro: cuota (FK), proveedor, id_transaccion, monto_cobrado, fecha_cobro, respuesta_raw (JSONField).

8. Tecnologias recomendadas

Componente	Tecnologia recomendada
Proveedor de pagos	Mercado Pago (preferido en Argentina) o Stripe. Ambos tienen SDK para Python y JS.
WhatsApp	Twilio WhatsApp API. Alternativa: Meta Cloud API (mas economica a escala).
Email	SendGrid o Django-anymail con Mailgun. Simple de integrar con Django.
Tareas periodicas	Celery + Redis (o Django-Q si se prefiere algo mas simple).
Tokens seguros	uuid.uuid4() + campo DateTimeField para expiracion. Opcional: itsdangerous para firmar.

9. Proximos pasos sugeridos

Paso 1	Disenar los modelos Contrato, Cuota y PagoRegistro en Django y correr las migraciones.
Paso 2	Crear la vista publica /pagar// con su logica de validacion de token.
Paso 3	Integrar el SDK de Mercado Pago en el frontend React para tokenizar tarjetas.
Paso 4	Implementar el endpoint /api/pagos/webhook/ y verificar la firma del proveedor.
Paso 5	Configurar Celery beat para generacion mensual de cuotas y marcado diario de vencidos.
Paso 6	Integrar Twilio o SendGrid para el envio automatico del link mensual.